Commit 53f00ae



apaszke committed on Aug 19, 2016

Add autograd

main · whc_flight_4 bc2caa7fdf006894eff7af936babde69ab5a40f8-huydhn-debug

1 parent 78a958a commit 53f00ae

11 files changed +565 -0 lines changed

Search within code

torch/autograd/__init__.py +3

```
@@ -0,0 +1,3 @@
+
+ from .variable import Variable
+ from .function import Function
```

torch/autograd/engine.py +80

```
@@ -0,0 +1,80 @@
+ from collections import Counter
+
+ class ExecutionEngine(object):
+     def __init__(self):
+         pass
+
+     def _compute_dependencies(self, function):
+         dependencies = {}
+         seen = {function}
```

```
+         queue = [function]
+         while len(queue) > 0:
+             fn = queue.pop()
+             for prev_fn, arg_id in fn.previous_functions:
+                 if prev_fn not in dependencies:
+                     dependencies[prev_fn] = [Counter() for _ in prev_fn.output_ids]
+                 output_idx = prev_fn.output_ids[arg_id]
+                 dependencies[prev_fn][output_idx][fn] += 1
+                 if prev_fn not in seen:
+                     queue.append(prev_fn)
+                     seen.add(prev_fn)
+         return dependencies
+
+     def _free_backward_dependency(self, dependencies, prev_fn, fn, arg_id):
+         deps = dependencies[prev_fn]
+         output_idx = prev_fn.output_ids[arg_id]
+         output_deps = deps[output_idx]
+         output_deps[fn] -= 1
+         if output_deps[fn] == 0:
+             del output_deps[fn]
+         return output_idx
+
+
+     def _is_ready_for_backward(self, dependencies, function):
+         for deps in dependencies[function]:
+             if len(deps) > 0:
+                 return False
+         return True
+
+     def run_backward(self, variable, grad):
+         ready = [(variable.creator, (grad,))]
+         not_ready = {}
+
+         dependencies = self._compute_dependencies(variable.creator)
+
+         while len(ready) > 0:
+             fn, grad = ready.pop()
```

```
47 +            # TODO: double-buffering
48 +            grad_input = fn._do_backward(*grad)
49 +            for (prev_fn, arg_id), d_prev_fn in zip(fn.previous_functions,
   grad_input):
50 +                if not prev_fn.requires_grad:
51 +                    assert d_prev_fn is None
52 +                    continue
53 +                output_nr = self._free_backward_dependency(dependencies,
   prev_fn, fn, arg_id)
54 +                is_ready = self._is_ready_for_backward(dependencies,
   prev_fn)
55 +                if is_ready:
56 +                    if prev_fn in not_ready:
57 +                        prev_grad = not_ready[prev_fn]
58 +                        if not prev_grad[output_nr]:
59 +                            prev_grad[output_nr] = d_prev_fn
60 +                        else:
61 +                            prev_grad[output_nr].add_(d_prev_fn)
62 +                        del not_ready[prev_fn]
63 +                    else:
64 +                        assert output_nr == 0
65 +                        prev_grad = (d_prev_fn,)
66 +                    ready.append((prev_fn, prev_grad))
67 +                else:
68 +                    if prev_fn in not_ready:
69 +                        prev_grad = not_ready[prev_fn]
70 +                    else:
71 +                        prev_grad = [None for _ in prev_fn.output_ids]
72 +
73 +                    if not prev_grad[output_nr]:
74 +                        prev_grad[output_nr] = d_prev_fn
75 +                    else:
76 +                        prev_grad[output_nr].add_(d_prev_fn)
77 +
78 +                    not_ready[prev_fn] = prev_grad
79 +
80 +
```

torch/autograd/function.py +40

```
@@ -0,0 +1,40 @@
+ from collections import Counter
+ from .variable import Variable
+
+ class Function(object):
+
+     def __init__(self):
+         self.previous_functions = None
+         self.output_ids = None
+         self.needs_input_grad = None
+
+     def __call__(self, *input):
+         return self._do_forward(*input)
+
+     def _do_forward(self, *input):
+         unpacked_input = tuple(arg.data for arg in input)
+         raw_output = self.forward(*unpacked_input)
+         if not isinstance(raw_output, tuple):
+             raw_output = (raw_output,)
+         self.needs_input_grad = tuple(arg.creator.requires_grad for arg in input)
+         self.requires_grad = any(self.needs_input_grad)
+         output = tuple(Variable(tensor, self) for tensor in raw_output)
+
+         self.previous_functions = [(arg.creator, id(arg)) for arg in input]
+         self.output_ids = {id(var): i for i, var in enumerate(output)}
+         return output
+
+     def _do_backward(self, grad_output, buffers=None):
+         grad_input = self.backward(grad_output)
+         if not isinstance(grad_input, tuple):
+             grad_input = (grad_input,)
+
+         assert len(grad_input) == len(self.previous_functions), \
+             self.__class__.__name__ + ' returned an invalid number of gradient tensors'
+         return grad_input
```

```
35 +
36 +     def forward(self, *input):
37 +         raise NotImplementedError
38 +
39 +     def backward(self, *grad_output):
40 +         raise NotImplementedError
```

torch/autograd/functions/TorchNode.py +33

```
... @@ -0,0 +1,33 @@
1  + import torch
2  + from ..base.Variable import Variable
3  + from ..base.Node import Node
4  +
5  + class TorchNode(Node):
6  +
7  +     def __init__(self, input_variable, *args):
8  +         self.input_variable = input_variable
9  +         # TODO: check if Tensors aren't mixed with Variables
10 +         self.args = args
11 +         self.unpacked_args = tuple(self._unpack_arg(arg) for arg in args)
12 +
13 +     def _forward(self):
14 +         result = getattr(self.input_variable.data, self.fn_name)
   (*self.unpacked_args)
15 +         # If a function returns a number, we have to wrap it again
16 +         if not torch.isTensor(result):
17 +             result = self.input_variable.new((result,))
18 +         return Variable(result, self)
19 +
20 +     def _backward(self, grad_output, *args, **kwargs):
21 +         grad_input = self.backward(grad_output)
22 +         if not isinstance(grad_input, tuple):
23 +             grad_input = (grad_input,)
24 +         variables = (self.input_variable,) + tuple(filter(lambda x:
   isinstance(x, Variable), self.args))
25 +         assert isinstance(grad_input, tuple)
26 +         assert len(variables) == len(grad_input)
27 +         for var, d_var in zip(variables, grad_input):
```

```
28 +             var.backward(d_var, *args, **kwargs)
29 +
30 +     def _unpack_arg(self, arg):
31 +         if isinstance(arg, Variable):
32 +             return arg.data
33 +         return arg
```

torch/autograd/functions/__init__.py +3

```
@@ -0,0 +1,3 @@
1 + from .basic_ops import *
2 + from .tensor import *
3 + from .pointwise import *
```

torch/autograd/functions/all_reduce.py +29

```
@@ -0,0 +1,29 @@
1  + from ..TorchNode import TorchNode
2  + from ...base.Variable import Variable
3  +
4  + class Sum(TorchNode):
5  +     fn_name = 'sum'
6  +
7  +     def backward(self, grad_output):
8  +         i = self.input_variable.data
9  +         if len(self.args) == 0:
10 +             return i.new(i.size()).fill_(grad_output[0]),
11 +         elif len(self.args) == 1:
12 +             dim = self.args[0]
13 +             repeats = [1 for i in range(i.dim())]
14 +             repeats[dim] = i.size(dim)
15 +             return grad_output.repeatTensor(*repeats),
16 +
17 + class Mean(TorchNode):
18 +     fn_name = 'mean'
19 +
20 +     def backward(self, grad_output):
21 +         i = inputs[0]
22 +         if len(self.args) == 0:
```

```
23 +             return i.new(i.size()).fill_(float(grad_output[0])/i.numel()),
24 +         elif len(self.args) == 1:
25 +             dim = self.args[0]
26 +             repeats = [1 for i in range(i.dim())]
27 +             repeats[dim] = i.size(dim)
28 +             return grad_output.repeatTensor(*repeats).div_(i.size(dim)),
29 +
```

torch/autograd/functions/basic_ops.py +145

```
@@ -0,0 +1,145 @@
1  + import torch
2  + from ..variable import Variable
3  + from ..function import Function
4  +
5  +
6  + class Add(Function):
7  +
8  +     def forward(self, a, b):
9  +         return a.add(b)
10 +
11 +     def backward(self, grad_output):
12 +         return grad_output, grad_output
13 +
14 +
15 + class Sub(Function):
16 +
17 +     def forward(self, a, b):
18 +         return a.sub(b)
19 +
20 +     def backward(self, grad_output):
21 +         return grad_output, grad_output.neg()
22 +
23 +
24 + class Mul(Function):
25 +
26 +     def forward(self, a, b):
27 +         self.input = (a, b)
28 +         return a.mul(b)
```

```
+
+    def backward(self, grad_output):
+        return grad_output.mul(self.input[1]),
  grad_output.mul(self.input[0])
+
+
+ class Div(Function):
+
+    def forward(self, a, b):
+        self.input = (a, b)
+        return a.div(b)
+
+    def backward(self, grad_output):
+        a, b = self.input
+        return grad_output.div(b),
  grad_output.neg().mul(a).div_(b).div_(b)
+
+ class Pow(Function):
+
+    def forward(self, a, b):
+        self.input = (a, b)
+        return a.pow(b)
+
+    def backward(self, grad_output):
+        a, b = self.input
+        return grad_output.mul(b).mul_(a.pow(b-1)),
  grad_output.mul(a.pow(b)).mul_(a.log())
+
+ class AddConstant(Function):
+
+    def __init__(self, constant):
+        self.constant = constant
+
+    def forward(self, a):
+        return a.add(self.constant)
+
+    def backward(self, grad_output):
+        return grad_output
+
```

```
65  +
66  + class SubConstant(Function):
67  +
68  +     def __init__(self, constant, sub_tensor=False):
69  +         self.constant = constant
70  +         self.sub_tensor = sub_tensor
71  +
72  +     def forward(self, a):
73  +         if self.sub_tensor:
74  +             return a.new().resizeAs_(a).fill_(self.constant).sub_(a)
75  +         else:
76  +             return a.sub(self.constant)
77  +
78  +     def backward(self, grad_output):
79  +         if self.sub_tensor:
80  +             return grad_output.neg()
81  +         else:
82  +             return grad_output
83  +
84  +
85  + class MulConstant(Function):
86  +
87  +     def __init__(self, constant):
88  +         self.constant = constant
89  +
90  +     def forward(self, a):
91  +         return a.mul(self.constant)
92  +
93  +     def backward(self, grad_output):
94  +         return grad_output.mul(self.constant)
95  +
96  +
97  + class DivConstant(Function):
98  +
99  +     def __init__(self, constant, div_by_tensor=False):
100 +         self.constant = constant
101 +         self.div_by_tensor = div_by_tensor
102 +
```

```
103 +     def forward(self, a):
104 +         if self.div_by_tensor:
105 +             self.input = a
106 +             return a.new().resizeAs_(a).fill_(self.constant).div_(a)
107 +         else:
108 +             return a.div(self.constant)
109 +
110 +     def backward(self, grad_output):
111 +         if self.div_by_tensor:
112 +             a = self.input
113 +             return grad_output.neg().mul_(self.constant).div_(a).div_(a)
114 +         else:
115 +             return grad_output.div(self.constant)
116 +
117 +
118 + class PowConstant(Function):
119 +
120 +     def __init__(self, constant, tensor_power=False):
121 +         self.constant = constant
122 +         self.tensor_power = tensor_power
123 +
124 +     def forward(self, a):
125 +         if self.tensor_power:
126 +             self.fw_result = torch.pow(self.constant, a)
127 +             return result
128 +         else:
129 +             self.input = a
130 +             return a.pow(self.constant)
131 +
132 +     def backward(self, grad_output):
133 +         if self.tensor_power:
134 +             return
    grad_output.mul(self.fw_result).mul_(math.log(self.constant))
135 +         else:
136 +             a = self.input
137 +             return
    grad_output.mul(self.constant).mul_(a.pow(self.constant-1))
138 +
139 + class Negate(Function):
```

```
140 +
141 +     def forward(self, i):
142 +         return i.neg()
143 +
144 +     def backward(self, grad_output):
145 +         return grad_output.neg()
```

torch/autograd/functions/pointwise.py +30

```
@@ -0,0 +1,30 @@
1  + from ..variable import Variable
2  + from ..function import Function
3  +
4  + class Exp(Function):
5  +
6  +     def forward(self, i):
7  +         self.result = i.exp()
8  +         return self.result
9  +
10 +     def backward(self, grad_output):
11 +         return self.result * grad_output
12 +
13 + class Log(Function):
14 +
15 +     def forward(self, i):
16 +         self.input = i
17 +         return i.log()
18 +
19 +     def backward(self, grad_output):
20 +         return grad_output.div(self.input)
21 +
22 + class Log1p(Function):
23 +
24 +     def forward(self, i):
25 +         self.input = i
26 +         return i.log1p()
27 +
28 +     def backward(self, grad_output):
29 +         return grad_output.div(self.input.add(1))
```

30 +

torch/autograd/functions/tensor.py +56

```
@@ -0,0 +1,56 @@
+ from ..function import Function
+ from ..variable import Variable
+
+ class Index(Function):
+
+     def __init__(self, *index):
+         self.index = index
+
+     def forward(self, i):
+         self.input_size = i.size()
+         return i[self.index]
+
+     def backward(self, grad_output):
+         # TODO: this won't have to be zeroed
+         grad_input = grad_output.new(self.input_size).zero_()
+         grad_input[self.index].copy_(grad_output)
+         return grad_input
+
+ class Transpose(Function):
+
+     def __init__(self, *dims):
+         assert len(dims) == 2
+         self.dims = dims
+
+     def forward(self, i):
+         return i.transpose(*self.dims)
+
+     def backward(self, grad_output):
+         return grad_output.transpose(*self.dims)
+
+ class View(Function):
+
+     def __init__(self, *sizes):
+         self.sizes = sizes
```

```
35 +
36 +     def forward(self, i):
37 +         self.input_size = i.size()
38 +         return i.view(*self.sizes)
39 +
40 +     def backward(self, grad_output):
41 +         # TODO: not sure if this clone is necessary
42 +         return grad_output.clone().view(self.input_size)
43 +
44 + class Copy(Function):
45 +
46 +     def __init__(self, dest_type):
47 +         self.dest_type = dest_type
48 +
49 +     def forward(self, i):
50 +         assert self.dest_type != type(i)
51 +         self.input_type = type(i)
52 +         return i.type(self.dest_type)
53 +
54 +     def backward(self, grad_output):
55 +         return grad_output.type(self.input_type)
56 +
```

torch/autograd/leaf.py +18

```
... @@ -0,0 +1,18 @@
1  + from collections import Counter
2  + from .function import Function
3  +
4  + class Leaf(Function):
5  +
6  +     def __init__(self, variable, requires_grad):
7  +         self.variable = variable
8  +         self.output_ids = {id(variable): 0}
9  +         self.previous_functions = []
10 +         self.requires_grad = requires_grad
11 +
12 +     def _do_forward(self, *input):
13 +         raise NotImplementedError
```

```
14 +
15 +     def _do_backward(self, *grad_output):
16 +         assert len(grad_output) == 1
17 +         self.variable.grad.add_(grad_output[0])
18 +         return tuple()
```

torch/autograd/variable.py +128

```
@@ -0,0 +1,128 @@
1  + from .engine import ExecutionEngine
2  +
3  + class Variable(object):
4  +
5  +     _execution_engine = ExecutionEngine()
6  +
7  +     _fallthrough_methods = [
8  +         'size',
9  +         'stride',
10 +         'nElement',
11 +         'numel',
12 +         # TODO: add more
13 +     ]
14 +
15 +     def __init__(self, tensor, creator=None, requires_grad=True):
16 +         if creator is None:
17 +             creator = Leaf(self, requires_grad)
18 +         self.data = tensor
19 +         self.creator = creator
20 +         self._grad = None
21 +
22 +     @property
23 +     def grad(self):
24 +         if self.creator.requires_grad:
25 +             # TODO: this won't have to be zeroed in the future
26 +             self._grad = self._grad or
   self.data.new(self.data.size()).zero_()
27 +         return self._grad
28 +
29 +     def __getattr__(self, name):
```

```
30 +         if name in self._fallthrough_methods:
31 +             return getattr(self.data, name)
32 +         raise AttributeError(name)
33 +
34 +     def __getitem__(self, key):
35 +         return Index(key)(self)[0]
36 +
37 +     def backward(self, gradient=None):
38 +         if gradient is None:
39 +             if self.data.numel() != 1:
40 +                 raise RuntimeError('backward should be called only on a
   scalar (i.e. 1-element tensor) or with gradient w.r.t. the variable')
41 +             gradient = self.data.new(1).fill_(1)
42 +         self._execution_engine.run_backward(self, gradient)
43 +
44 +     def __repr__(self):
45 +         return 'Variable containing:' + self.data.__repr__()
46 +
47 +     def contiguous_(self):
48 +         self.data = self.data.contiguous()
49 +         return self
50 +
51 +     def type(self, t):
52 +         if t != type(self.data):
53 +             return Copy(t)(self)[0]
54 +         return self
55 +
56 +     def add(self, other):
57 +         if isinstance(other, Variable):
58 +             return Add()(self, other)[0]
59 +         else:
60 +             return AddConstant(other)(self)[0]
61 +
62 +     def sub(self, other):
63 +         if isinstance(other, Variable):
64 +             return Sub()(self, other)[0]
65 +         else:
66 +             return SubConstant(other)(self)[0]
67 +
```

```
68  +     def mul(self, other):
69  +         if isinstance(other, Variable):
70  +             return Mul()(self, other)[0]
71  +         else:
72  +             return MulConstant(other)(self)[0]
73  +
74  +     def div(self, other):
75  +         if isinstance(other, Variable):
76  +             return Div()(self, other)[0]
77  +         else:
78  +             return DivConstant(other)(self)[0]
79  +
80  +     def pow(self, other):
81  +         if isinstance(other, Variable):
82  +             return Pow()(self, other)[0]
83  +         else:
84  +             return PowConstant(other)(self)[0]
85  +
86  +     def view(self, *sizes):
87  +         return View(*sizes)(self)[0]
88  +
89  +     def t(self):
90  +         return Transpose(0, 1)(self)[0]
91  +
92  +     def transpose(self, dim1, dim2):
93  +         return Transpose(dim1, dim2)(self)[0]
94  +
95  +     def __add__(self, other):
96  +         return self.add(other)
97  +     __radd__ = __add__
98  +
99  +     def __sub__(self, other):
100 +         return self.sub(other)
101 +
102 +     def __rsub__(self, other):
103 +         return SubConstant(other, sub_tensor=True)(self)[0]
104 +
105 +     def __mul__(self, other):
```

```
106 +             return self.mul(other)
107 +         __rmul__ = __mul__
108 +
109 +     def __div__(self, other):
110 +         return self.div(other)
111 +     __truediv__ = __div__
112 +
113 +     def __rdiv__(self, other):
114 +         return DivConstant(other, div_by_tensor=True)(self)[0]
115 +     __rtruediv__ = __rdiv__
116 +
117 +     def __pow__(self, other):
118 +         return self.pow(other)
119 +
120 +     def __rpow__(self, other):
121 +         return PowConstant(other, tensor_power=True)(self)[0]
122 +
123 +     def __neg__(self):
124 +         return Negate()(self)[0]
125 +
126 +
127 + from .leaf import Leaf
128 + from .functions import *
```

Comments 0 (0)

Please sign in to comment.